LiteCDN: A Lightweight Content Delivery Network

Jimeet Shastri, Bhavishy Agrawal, Husain Mohammad Ali, Hayagriv Desikan
Department of Computer Science
Indian Institute of Technology, Jodhpur
{b24cs1034, b24cs1023, b24cs1084, b24cs1092}@iitj.ac.in

***Abstract*—LiteCDN is a lightweight experimental Content Delivery Network (CDN) designed to study routing and caching strategies in a controlled small-scale environment. The system consists of one origin server and six distributed edge servers coordinated through a centralized routing service. Request distribution evolved from baseline round-robin selection to an α–β weighted latency–load scoring model, extended with bounded parameter perturbation and ε-greedy exploration to prevent persistent edge dominance under burst traffic.**

Edge servers implement a segmented in-memory cache divided into New/Unproven (35%), Popular/Proven (60%), and Miss-Aware (5%) regions to improve structural stability and long-tail retention compared to a unified LRU approach. Evaluation was conducted using stress-based synthetic simulations to highlight behavioral differences between routing and caching policies rather than to benchmark real-world throughput.

Results demonstrate that weighted routing significantly reduces load concentration under sustained demand, while segmented caching enforces predictable eviction behavior with modest impact on aggregate hit rate. LiteCDN therefore serves as a structured experimental model for analyzing stability-oriented CDN design under controlled conditions.

***Index Terms*—Content Delivery Network, Edge Caching, Latency Reduction, Static File Delivery, Server Consistency**

I. Problem Statement

A. Objective

The primary objective of LiteCDN is to develop a lightweight content delivery network that minimizes latency in accessing static files while maintaining consistency across distributed edge servers with minimal hardware costs. The system aims to provide faster content delivery to end users through strategic edge server placement and intelligent caching mechanisms, without requiring expensive infrastructure typically associated with enterprise-level CDN solutions.

B. Motivation

With the exponential growth of web-based applications and increasing user expectations for instant content access, efficient content delivery has become crucial. Traditional CDN solutions, while effective, often require substantial capital investment and complex configurations that exceed the requirements and budgets of small to medium-scale deployments.

The motivation behind LiteCDN stems from the need to:

- Reduce file access latency for end users by bringing content closer to them geographically
- Minimize load on the origin server by distributing requests across multiple edge servers
- Maintain consistency across distributed servers to ensure users always receive the correct, up-to-date content
- Keep hardware and operational costs minimal, making CDN technology accessible to smaller organizations and educational projects
- Provide hands-on learning experience with distributed systems, caching strategies, and routing algorithms

The project addresses the challenge of building a functional CDN system that balances performance optimization with resource constraints, making it ideal for understanding core CDN concepts and implementation practices.

II. Review of Existing Technologies and Solutions

A. Architecture of Modern CDN Systems

Modern Content Delivery Networks (CDNs) such as Cloudflare, Amazon CloudFront, Akamai, Fastly, and Azure CDN operate large-scale distributed infrastructures consisting of globally deployed edge servers. These systems are designed around three core architectural layers:

- Edge Layer – geographically distributed servers that serve cached content
- Routing Layer – intelligent request steering system
- Origin Layer – authoritative source of content

When a client requests a resource, DNS-based routing or anycast routing directs the request to the nearest or lowest-latency edge node. The edge node performs a cache lookup. If the content exists (cache hit), it is served locally. Otherwise (cache miss), the request is forwarded to the origin server, the response is cached, and then delivered to the client.

The effectiveness of a CDN is therefore determined primarily by three algorithmic components:

1) Request routing algorithm
2) Cache placement and eviction strategy
3) Load balancing policy

B. Routing Algorithms Used in Commercial CDNs

1) **Anycast Routing (Cloudflare, Fastly)**

Anycast routing assigns the same IP address to multiple geographically distributed servers. Internet routing protocols automatically direct client traffic to the nearest available server based on network topology.

Advantages

- Extremely low latency due to shortest network path selection

- Automatic failover if a node goes offline
- No client-side logic required

Limitations

- Limited control over traffic distribution
- Routing decisions depend on ISP BGP policies
- Difficult to guarantee deterministic server selection

—

2) DNS-Based Latency Routing (Amazon CloudFront, Azure CDN)

DNS-based routing selects an edge server based on latency measurements, geographic location, or server health.

Selection logic:

```
choose server S such that
latency(client, S) is minimum
```

Advantages

- More controllable than anycast
- Can consider server load and health
- Enables traffic engineering

Limitations

- DNS caching may delay routing updates
- Requires continuous telemetry measurements
- Increased control plane complexity

—

3) Region-Based Routing

Used in mid-scale CDN systems and research implementations, region-based routing maps users to nearest servers using IP geolocation databases.

Advantages

- Simple implementation
- Predictable routing behavior
- Lower computational overhead

Limitations

- Accuracy depends on IP database quality
- Does not adapt to real-time congestion
- Can cause uneven server load distribution

—

4) Round-Robin Routing

Requests are distributed cyclically across available servers.

Advantages

- Extremely simple
- Even request distribution under uniform traffic
- Zero monitoring overhead

Limitations

- Ignores latency and server health
- Performs poorly under heterogeneous server capacities
- Not suitable for geographically distributed systems

—

C. Caching Algorithms Used in Existing CDNs

Caching efficiency directly determines CDN performance. Real systems employ multiple eviction and expiration policies.

1) TTL-Based Expiration

Each cached object is stored with a Time-To-Live value. After expiration, it is considered stale and must be refreshed from origin.

Advantages

- Predictable consistency guarantees
- Minimal computation overhead
- Easy implementation

Limitations

- May serve stale content before expiration
- Requires manual tuning
- Cannot adapt to changing access patterns

—

2) Least Recently Used (LRU)

Removes items that have not been accessed recently.

Advantages

- Effective for workloads following Zipf distribution
- Maximizes cache hit probability
- Memory efficient

Limitations

- Computational overhead for maintaining access order
- Poor performance for burst access patterns
- Cannot distinguish between popular and recently accessed content

—

3) Least Frequently Used (LFU)

Evicts content accessed least frequently.

Advantages

- Retains popular objects
- Stable hit rate under steady workloads

Limitations

- Requires counters for each object
- Slow adaptation to changing traffic
- Memory overhead

—

4) Hierarchical Cache Systems

Enterprise CDNs implement multi-level caches:

```
Edge Cache -> Regional Cache ->
Core Cache -> Origin
```

Advantages

- Reduces origin load dramatically
- Improves hit rate for unpopular content
- Enables scalability to millions of requests/sec

Limitations

- Increased architectural complexity
- Higher synchronization overhead
- More difficult consistency management

—

D. Load Balancing Algorithms

Load balancing prevents individual servers from becoming bottlenecks.

Common strategies

- Least Connections — selects server with fewest active sessions
- Weighted Round Robin — distributes traffic proportionally to capacity
- Power of Two Choices — randomly samples two servers and selects the less loaded
- Consistent Hashing — assigns clients deterministically to servers

Engineering Trade-off

Sophisticated algorithms improve performance but require monitoring systems, telemetry collection, and control-plane logic.

—

E. Evaluation of Commercial CDN Platforms

Strengths

- Global infrastructure reduces latency worldwide
- Automatic failover and redundancy
- Advanced traffic optimization algorithms
- Integrated security features (DDoS mitigation, TLS termination)
- Elastic scaling to handle traffic spikes

Limitations

- High operational cost at scale
- Limited transparency of internal algorithms
- Vendor dependency and lock-in
- Configuration flexibility restricted by provider APIs
- Performance tuning requires platform-specific expertise

—

F. Evaluation of Open-Source CDN Components

Open-source tools such as Apache Traffic Server, Varnish, and NGINX provide individual CDN functionalities but do not constitute complete CDN systems.

Strengths

- Full control over caching policies
- Highly customizable routing logic
- No vendor lock-in
- Cost-effective deployment

Limitations

- Require manual integration of components
- Lack centralized orchestration layer
- Monitoring and failover must be implemented separately
- Operational complexity increases rapidly with scale

—

G. Identified Gap in Existing Solutions

Analysis of both commercial platforms and open-source systems reveals a gap between enterprise-grade CDN infrastructure and educational or small-scale deployment needs. Existing commercial solutions prioritize scalability and reliability at the cost of complexity and financial expense, while open-source tools prioritize flexibility but require significant engineering effort to integrate into a complete system.

Therefore, there exists a need for a lightweight CDN architecture that:

- Demonstrates core CDN principles clearly
- Requires minimal hardware resources
- Provides integrated routing, caching, and monitoring
- Maintains conceptual transparency for educational use
- Allows algorithmic experimentation and modification

Such systems serve as practical platforms for understanding distributed systems, network optimization, and large-scale web infrastructure design.

III. Solution Trajectory

A. System Architecture

The LiteCDN system adopts a modular, service-oriented architecture designed to simulate a small-scale static Content Delivery Network (CDN) while preserving controlled experimental complexity. The architecture separates request orchestration, routing intelligence, segmented edge caching, origin authority, and administrative observability.

Rather than targeting production-scale elasticity, the system prioritizes clarity, measurability, and reproducibility to enable structured evaluation of routing and caching strategies under stochastic request patterns.

1) Core Components:

a) CDNSystem: The **CDNSystem** functions as the central orchestrator, coordinating interactions between the RoutingService, multiple EdgeServer instances, and the OriginServer. It exposes distinct interfaces for client requests and administrative control, ensuring that runtime routing behavior remains isolated from configuration operations.

The system maintains references to all edge nodes and the origin server, enabling coordinated request handling without introducing inter-edge communication.

b) RoutingService: The **RoutingService** distributes incoming requests across six edge servers. Routing evolved from baseline round-robin selection to a weighted latency–load scoring model:

$$\text{Score} \;=\; \alpha \cdot \text{Latency} \;+\; \beta \cdot \text{Load}, \qquad \alpha + \beta = 1. \tag{1}$$

The model treats latency and load as weakly correlated under nominal conditions, while accounting for nonlinear coupling near saturation. Rather than pursuing gradient-based optimization, routing seeks to maintain operation within a stable region balancing responsiveness and utilization.

To prevent persistent dominance of a single edge, the routing logic incorporates:

- Bounded parameter perturbation under stagnation conditions;
- ε-greedy exploration, occasionally selecting the second-best edge.

This two-stage approach—deterministic scoring followed by controlled exploration—preserves stability while enabling limited state-space sampling. Routing remains centralized to simplify observability and reproducibility.

c) EdgeServer (6 Instances): Six **EdgeServer** instances represent distributed caching nodes across distinct regions. Each edge maintains an in-memory segmented cache consisting of:

- **New/Unproven** (35%) – probationary space for newly fetched objects, promoted only after repeated access;
- **Popular/Proven** (60%) – stable retention for frequently accessed content;
- **Miss-Aware** (5%) – constrained buffer for objects promoted based on repeated origin-level cache misses.

Cache operations include TTL-based expiration, LRU eviction within segments, and prioritized inter-segment eviction (New $\rightarrow$ Miss-Aware $\rightarrow$ Popular). This segmentation isolates transient content from stable hot objects while accommodating recurring long-tail requests without destabilizing popular content retention.

d) OriginServer: The **OriginServer** stores authoritative content and acts as the metadata authority for miss-aware promotion. It tracks repeated cache-miss fetches across edges and signals eligibility for placement in the miss-aware region when predefined thresholds are exceeded.

Centralizing miss metadata avoids distributed coordination complexity while enabling data-driven long-tail optimization.

e) CacheEntry and Segment: **CacheEntry** encapsulates file content, TTL, access metadata, and segment identity required for promotion and eviction decisions. Segment-level encapsulation ensures LRU enforcement and capacity constraints remain isolated within each region.

This abstraction allows experimentation with caching policies without modifying routing logic.

f) AdminAPI and SiteOwnerConsole: The **AdminAPI** exposes endpoints for origin configuration, health monitoring, and routing parameter adjustment. The **SiteOwnerConsole** provides lightweight system observability without introducing additional frontend complexity.

Administrative operations remain logically decoupled from request-serving paths to preserve routing stability.

2) Architectural Evolution Across Sprints: The architecture matured incrementally:

- **Sprint 1:** Established end-to-end request flow with baseline round-robin routing.
- **Sprint 2:** Introduced segmented caching and deterministic α–β weighted routing.
- **Sprint 3:** Added bounded perturbation, ε-greedy exploration, and administrative observability.
- **Sprint 4:** Focused on integration stabilization, testing, logging, and documentation finalization.

This sequencing ensured routing intelligence and cache segmentation were introduced only after foundational correctness was validated.

3) Design Rationale: The modular decomposition enables:

- Independent evolution of routing and caching policies;
- Controlled experimentation with latency–load trade-offs;
- Isolation of origin authority from edge caching logic;
- Stability-oriented routing rather than global optimization;
- Segment-based lifecycle separation for transient, popular, and miss-aware content.

Key architectural assumptions include weak latency–load correlation under nominal regimes, centralized routing for reproducibility, absence of distributed cache coherence, and bounded system scale sufficient to observe routing diversity without excessive complexity.

The system intentionally favors analytical clarity and structured experimentation over enterprise-scale deployment.

B. Identified Tech Stack

After evaluating multiple implementation options, the following technology stack was selected to balance architectural clarity, performance characteristics, and system complexity appropriate for a lightweight experimental CDN.

1) Backend Framework: Node.js with Express: The backend components (CDNSystem, RoutingService, EdgeServer, OriginServer, and AdminAPI) are implemented using Node.js with Express.

a) Rationale: The CDN workload is predominantly I/O-bound, involving network request handling, static file serving, and lightweight routing computations. Node.js employs an asynchronous, event-driven, non-blocking I/O model well-suited for concurrent network operations without the overhead of thread-per-request architectures. Since the system performs no CPU-intensive computation, the single-threaded event loop simplifies concurrency management while providing sufficient throughput.

Express provides a minimal abstraction layer with clear routing, middleware support for modular service composition, and straightforward REST API integration—enabling CDN components to simulate distributed edge behavior with implementation clarity.

b) Tradeoff Analysis: A Java/Spring-based solution was considered but rejected due to higher memory overhead from its thread-per-request model and configuration complexity disproportionate to the experimental scope. Python (Flask/FastAPI) was also evaluated; however, the Global Interpreter Lock (GIL) introduces concurrency limitations that could affect multi-request simulation fidelity. Node.js provides an effective balance between concurrency realism, I/O performance, and implementation simplicity.

2) Frontend Framework: React.js: The administrative interface (SiteOwnerConsole) is implemented using React.js.

a) Rationale: The admin console supports origin configuration, edge health monitoring, routing and cache behavior visualization, and is designed for extensibility. React was selected for its component-based architecture enabling modular UI development, scalable state management, reusable components across administrative views, and virtual DOM rendering efficiency for real-time system state updates.

b) Tradeoff Analysis: While a framework-based approach introduces build tooling and dependency management overhead compared to plain HTML/JavaScript, it provides superior long-term scalability, code organization, and integration with API-driven architectures. For a system intended to

evolve beyond a minimal dashboard, React offers a sustainable frontend foundation.

3) Storage and Caching Strategy: Caching is implemented using in-memory data structures (JavaScript `Map`/`Object`), while configuration persistence uses lightweight JSON files.

a) Rationale: Edge caches simulate fast-access memory typical of CDN edge nodes. Since the project focuses on routing and caching behavior evaluation rather than persistence durability, in-memory storage ensures constant-time access, minimal latency overhead, and clear visibility into cache state transitions. JSON files provide human-readable, version-trackable configuration with minimal setup requirements.

b) Tradeoff Analysis: Introducing an external store (e.g., Redis or MongoDB) would add network latency, operational complexity, and infrastructure dependencies unrelated to the experimental objectives. While in-memory storage lacks durability across restarts, persistence is not required for evaluating routing algorithms or cache segmentation strategies. This design isolates performance measurements to CDN logic rather than infrastructure layers.

4) Development Tools:

- **Git** – version control and collaborative development
- **Postman** – API testing and endpoint validation
- **npm** – dependency management for Node.js and React ecosystems

These tools were selected for ecosystem maturity, stability, and compatibility with the chosen architecture.

C. Sprint Plans and Scrum Teams

LiteCDN development followed an Agile life cycle organized into four two-week sprints structured around technical dependencies and incremental stabilization. Sprint 1 established the end-to-end request flow with baseline round-robin routing to validate connectivity across the client, *CDNSystem*, edge servers, and origin server. Sprint 2 introduced segmented edge caching with TTL/LRU eviction and transitioned to weighted routing using an α–β latency–load scoring model. Sprint 3 extended routing through bounded perturbation and ε-greedy exploration while adding administrative and monitoring capabilities via the *AdminAPI* and *SiteOwnerConsole*. Sprint 4 focused on integration testing, structured logging, error handling, and finalization of UML and SRS artifacts.

TABLE I
SPRINT GOALS AND KEY TECHNICAL TASKS

Sprint	Duration	Sprint Goal	Key Technical Tasks
Sprint 1	2 Weeks	Establish end-to-end request flow and connectivity	– Implement minimal OriginServer (serve static files) – Implement EdgeServer pass-through – Implement CDNSystem.handleRequest() – Implement RoutingService (Round Robin) – Basic console logging
Sprint 2	2 Weeks	Implement segmented caching and weighted routing	– Define CacheEntry structure with metadata, ttl, hitCount, segment – Implement Segment class with canAccept() and evictLRU() – Implement cacheSegments map (NewUnproven/Popular/MissAware) – Implement cache hit/miss logic in getFile() – Implement putCache() with segment selection – Implement TTL-based expiry with expireTTL() – Implement LRU eviction policy per segment – Add alpha, beta attributes to RoutingService – Implement calculateScore(latency, load) method – Implement latencyMap and loadMap tracking – Basic unit tests for routing and cache logic
Sprint 3	2 Weeks	Add routing optimization and administrative control	– Implement selectEdge(epsilon) with ε-greedy logic – Implement perturbParameters() with stagnation detection – Add stagnationCounter to RoutingService – Implement updateMetrics() for routing feedback – Implement trackMiss() at OriginServer – Add missTracker and getMissCount() – Implement AdminAPI interface (updateOriginConfig, getHealthMetrics, updateRoutingParams, viewEdgeHealth) – Implement health endpoints (updateLoad, healthCheck) – Implement SiteOwnerConsole (CLI or minimal UI) – Integrate health metrics collection
Sprint 4	2 Weeks	Stabilize system and finalize documentation	– Implement unit and integration tests for routing-caching interactions – Improve error handling (404/timeouts/network failures) – Structured logging enhancements for cache operations – Performance optimization of segment lookups – Finalize UML diagrams (Class, Sequence, Activity, Component) – Update SRS with all method signatures – Update Traceability Matrix – Document architectural assumptions and design rationale

TABLE II
SPRINT DELIVERABLES AND ROLE FOCUS

Sprint	Deliverables	Role Focus & Work Assigned
Sprint 1	Working request path: Client → CDN → Edge → Origin	PO: Define connectivity backlog, validate end-to-end flow. SM: Conduct sprint planning and daily syncs, remove blockers. Developer: Design server skeleton and define core modules. Developer: Validate request flow and update sequence diagram.
Sprint 2	Functional edge cache with segmented structure and weighted routing	PO: Prioritize caching features, validate hit/miss scenarios. SM: Coordinate routing-cache integration, monitor sprint progress. Developer: Implement segmented cache with TTL and LRU eviction. Developer: Add basic cache test cases and update documentation.
Sprint 3	Admin interface with routing optimization and system observability	PO: Define AdminAPI backlog, validate configuration and health features. SM: Facilitate AdminAPI integration and routing enhancements. Developer: Implement AdminAPI endpoints and health checks.
Sprint 4	Tested system + finalized documentation	PO: Validate feature completeness, approve final readiness. SM: Ensure Definition of Done and integration stability. Developer: Perform integration testing and finalize documentation.

D. Alpha Managers and Responsibility Allocation

To ensure systematic maturity across project dimensions, responsibility ownership was structured using the Alpha concept. Rather than distributing responsibilities across multiple overlapping roles, two consolidated Alpha Managers were designated to maintain clarity and reduce coordination overhead.

1) Alpha Progression Across Sprints: As shown in Table I, the progression of key project aspects—Requirements, Architecture, Implementation, Testing, and Documentation—was tracked across all four sprints to ensure balanced development maturity. Table I presents the Alpha state progression observed during the sprint lifecycle.

2) Alpha Manager – Architecture, UML, and Implementation: This role oversees the structural and technical integrity of the system, including:

- Defining and maintaining overall system architecture
- Ensuring consistency across UML artifacts (Class, Sequence, Component, and Deployment diagrams)
- Validating alignment between design and implementation
- Supervising core module development

By consolidating architectural and implementation oversight, this role ensured that evolving features remained structurally coherent throughout the sprint lifecycle.

3) Alpha Manager – Requirements, Documentation, and Integration: This role ensures traceability and requirement fulfillment by:

- Maintaining the Software Requirements Specification (SRS)
- Updating and validating functional and non-functional requirements
- Managing documentation artifacts and traceability matrices
- Overseeing integration validation against defined requirements

This consolidation reduced fragmentation between documentation and integration processes, ensuring implemented features were continuously evaluated against requirement intent.

TABLE III
ALPHA STATES ACROSS SPRINTS

Alpha	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Requirements	Draft	In Progress	Reviewed	Done
Architecture	Draft	In Progress	Reviewed	Done
Implementation	-	Draft	In Progress	Done
Testing	-	-	Draft	Done
Documentation	Draft	In Progress	In Progress	Done

E. Alpha Card Progression

The alpha progression tracks the maturity of key project aspects across sprints:

Requirements Alpha:

- Sprint 1: Initial requirements drafted, basic use cases identified
- Sprint 2: Requirements refined based on technical feasibility
- Sprint 3: Requirements reviewed against implementation
- Sprint 4: Final validation and sign-off

Architecture Alpha:

- Sprint 1: High-level architecture design, component identification
- Sprint 2: Detailed design of caching mechanism and routing logic
- Sprint 3: Architecture validation through implementation
- Sprint 4: Final UML diagrams and architecture documentation

Implementation Alpha:

- Sprint 2: Core functionality implementation begins
- Sprint 3: Feature completion and integration
- Sprint 4: Bug fixes, optimization, and finalization

Testing Alpha:

- Sprint 3: Unit tests for individual components
- Sprint 4: Integration tests, performance tests, and bug fixes

F. Coding Methodology and Protocols

To ensure code quality, maintainability, and effective team coordination, a structured set of development protocols was adopted throughout the project lifecycle.

1) Branching Strategy: A disciplined Git workflow was followed to separate stable releases from ongoing development:

- **Main Branch**: Contains only stable, tested, and deployable code.
- **Development Branch**: Serves as the primary integration branch for validated features.
- **Feature Branches**: Created using the naming pattern *feature/feature-name*.

- **Bug Fix Branches**: Created using the naming pattern *bugfix/issue-description*.
- Merges into the main branch were permitted only after successful code review and testing.

2) Code Review Process: All changes were validated through a lightweight peer-review mechanism:

- Every pull request required approval from at least one team member.
- Reviewers evaluated correctness, functionality, readability, and standards compliance.
- A standardized checklist was used, covering:
  - Functional correctness
  - Code style and consistency
  - Error handling robustness
  - Adequacy of test coverage
- All review comments were addressed prior to merging.

3) Naming Conventions: Consistent naming conventions were enforced to improve readability and maintainability:

- **Classes**: PascalCase (e.g., *EdgeServer*, *CacheEntry*, *RoutingService*)
- **Methods and Variables**: camelCase (e.g., *routeRequest*, *cacheHitCount*)
- **Constants**: UPPER_SNAKE_CASE (e.g., *MAX_CACHE_SIZE*, *DEFAULT_TTL*)
- **Files**: Lowercase with hyphens (e.g., *edge-server.js*, *routing-service.js*)

4) Commit Message Convention: Commit messages followed a semantic structure to enhance traceability:

- **feat**: Introduction of new features
- **fix**: Bug fixes
- **docs**: Documentation updates
- **test**: Test additions or modifications
- **refactor**: Code restructuring without functional changes
- **style**: Formatting or stylistic changes

Examples include: *"feat: add cache hit tracking"*, *"fix: resolve routing logic error"*.

5) Testing Standards: A multi-layered testing strategy was implemented:

- Minimum 70% code coverage for core modules
- Unit tests for utility functions and core classes
- Integration tests for API endpoints
- Manual validation checklist for UI components

IV. Results and Discussion

A. Evaluation Methodology

To clearly observe structural differences between routing and caching strategies, LiteCDN was evaluated under controlled high-stress conditions. The objective was not to emulate realistic Internet latency, but to amplify behavioral differences between algorithms within a reproducible experimental environment.

Six abstract edge servers were modeled, each assigned a baseline latency between 50 and 150 ms. Latency increased proportionally with server load according to:

$$Latency = Baseline + 80 \cdot Load \tag{2}$$

Load accumulated when a server was selected and decayed minimally to simulate burst-like traffic. A total of 3000 sequential requests were issued to impose sustained pressure on the routing system.

For cache evaluation, 3000 requests were generated using a Zipf distribution (parameter = 1.3) across 400 objects to represent highly skewed access patterns typical of web workloads.

The following performance metrics were collected:

- Average latency
- Load variance across edge servers
- Maximum observed edge load
- Cache hit ratio

The evaluation emphasized structural behavior and tradeoffs under stress rather than average-case performance.

B. Routing Strategy Evaluation

Five routing strategies were evaluated:

- Round-robin
- Pure latency-based selection
- Pure load-based selection
- α–β weighted routing ($\alpha = 0.5$, $\beta = 0.5$)
- α–β routing with ε-greedy exploration ($\varepsilon = 0.25$)

1) Load Distribution: Under high-stress conditions, significant differences were observed in load variance across edge servers.

Pure latency routing produced a load variance of approximately 0.402, indicating strong concentration on the fastest node. In contrast, round-robin and pure load routing maintained low variance (approximately 0.073), reflecting uniform distribution without performance-awareness.

The α–β weighted strategy reduced variance to approximately 0.145, demonstrating that combining latency and load mitigates overload concentration. Incorporating ε-greedy exploration further stabilized distribution (variance ≈ 0.142) by preventing persistent node dominance.

Although pure latency routing increased load concentration, the maximum observed load across strategies remained similar (approximately 158–159 in simulation scale). Under sustained burst traffic, demand gradually spread across edges as latency penalties discouraged continued concentration. Consequently, routing strategies primarily influenced distribution fairness rather than peak load magnitude.

C. Cache Policy Evaluation

Cache behavior was evaluated by comparing a standard LRU policy with the segmented 35–60–5 design under heavy skew.

The LRU policy achieved a hit ratio of approximately 0.757, whereas the segmented policy achieved approximately 0.704.

While LRU yielded a higher hit rate, the segmented design introduced structural separation between new content, stable popular objects, and moderate-frequency items. This reduced

TABLE IV
ARCHITECTURAL REFINEMENTS ACROSS DEVELOPMENT SPRINTS

Feature / Component	Initial Plan	Final Implementation	Change Type	Rationale (Reduced)
Routing Algorithm	Round-robin	α–β weighted scoring + ε-greedy exploration	Modified	Round-robin ignored latency/load variation; weighted scoring reduced overload under burst traffic.
Routing Parameters	Static weights	Bounded perturbation with stagnation detection	Added	Static weights caused routing stagnation; perturbation maintained adaptability without instability.
Cache Policy	LRU cache	Segmented cache (35% New, 60% Popular, 5% Miss-Aware)	Modified	LRU evicted moderate-frequency objects; segmentation improved long-tail retention.
Segment Allocation	Equal split	Tuned allocation (35/60/5)	Modified	Equal split underutilized popular region; tuning improved cache stability.
Miss Tracking	Local tracking	Centralized origin-level miss tracking	Added	Local tracking lacked global visibility; origin aggregation enabled controlled promotion.
Cache Memory Control	Implicit sizing	Strict capacity limits + TTL enforcement	Modified	Unbounded growth caused instability; limits stabilized behavior.
Edge Coordination	Inter-edge sync	Removed (centralized routing retained)	Removed	Coordination added complexity without measurable benefit at small scale.
Admin Observability	Basic endpoints	Health metrics + routing parameter adjustment	Extended	Evaluation required runtime observability and parameter tuning.

frequent eviction of moderately popular objects during transient request bursts.

The segmented policy therefore sacrifices approximately five percentage points of hit rate in exchange for more stable and predictable eviction behavior, consistent with the design objective of controlled cache dynamics.

D. Latency–Load Relationship Validation

The stress model confirmed that latency increases significantly with load accumulation. Under pure latency routing, this effect amplified concentration on initially fast nodes. The α–β strategy penalized high-load nodes earlier, reducing imbalance before saturation effects intensified.

These observations support the use of linear weighted scoring combined with bounded exploration rather than more complex optimization mechanisms.

E. Observed Tradeoffs

Several tradeoffs emerged during evaluation:

- Centralized routing simplifies control but may limit scalability
- ε-greedy exploration increases short-term variability
- Segmented caching reduces raw hit rate under heavy skew
- Fixed segment ratios limit dynamic adaptability

These tradeoffs were considered acceptable to preserve architectural clarity and experimental control.

F. Overall Findings

Under sustained high-load conditions, the evaluation indicates:

- Pure latency routing leads to significant load concentration
- Weighted routing substantially reduces imbalance
- Controlled exploration improves long-term fairness
- Segmented caching enforces structural stability with modest hit-rate reduction

Overall, LiteCDN achieves its intended objective as a structured experimental CDN model emphasizing balanced behavior and stability under stress rather than pure optimization.

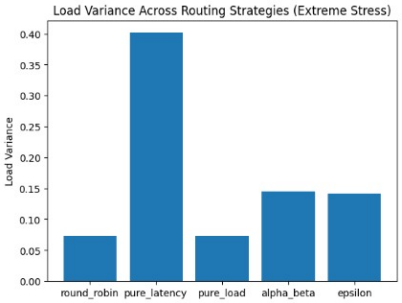

Fig. 1. Load Variance Across Routing Strategies in High-Stress Conditions

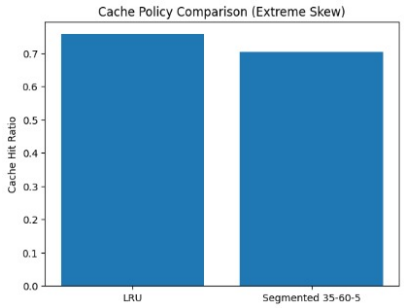

Fig. 2. Cache Policy Comparison (Extreme Skew)

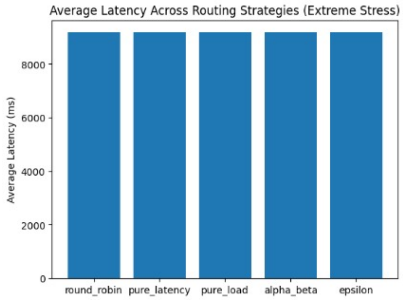

Fig. 3. Average Latency Across Routing Strategies Under Load

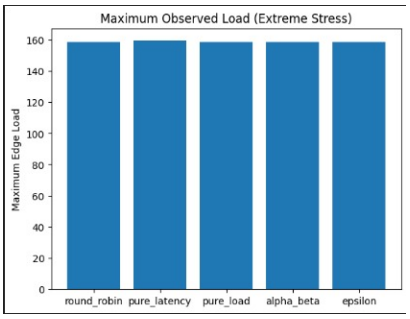

Fig. 4. Maximum Observed Load under Extreme Stress Conditions

V. Conclusion

LiteCDN successfully demonstrates a functional content delivery network that achieves the primary objectives of reducing file access latency and maintaining server consistency with minimal hardware requirements. The project delivered a working system with six edge servers and one origin server, implementing intelligent weighted routing with exploration and segmented caching mechanisms.

Key Achievements:

- Successful implementation of weighted α–β routing with ε-greedy exploration
- Three-tier cache segmentation (NewUnproven/Popular/MissAware) improving retention of popular content
- Functional admin interface for monitoring, health metrics, and routing parameter adjustment
- Comprehensive documentation including SRS, UML diagrams, and architectural rationale

Learning Outcomes:

The project provided hands-on experience with distributed systems architecture, adaptive routing algorithms, and multi-tier caching strategies. The team gained practical knowledge in:

- Agile/Scrum methodology with sprint-based incremental development and alpha progression tracking
- Git-based collaborative development workflows with feature branching and code review
- Node.js asynchronous programming patterns for handling concurrent requests
- System architecture design with modular service-oriented decomposition
- Adaptive algorithm design including ε-greedy exploration and bounded parameter perturbation
- Trade-off analysis between cache consistency, hit rates, and memory utilization
- Structured experimentation methodology for evaluating algorithmic alternatives

Technical Insights:

Several technical challenges provided valuable insights:

- Weighted routing required adaptive parameter tuning to maintain effectiveness under varying load conditions
- Centralized miss tracking at origin enabled targeted cache promotion without distributed coordination overhead
- ε-greedy exploration prevented routing stagnation while maintaining overall performance
- Bounded perturbation successfully adapted routing parameters without oscillatory instability

Challenges Overcome:

The team successfully addressed concurrent request handling through asynchronous patterns, cache consistency through invalidation APIs and miss tracking, memory management through segment-specific eviction, segment selection through graduated promotion heuristics, and routing parameter optimization through adaptive perturbation and exploration.

Future Enhancements:

Potential improvements for future versions include:

- Support for dynamic content delivery with cache validation
- Machine learning-based adaptive TTL based on content access patterns
- Automatic cache invalidation using webhooks or publish-subscribe mechanisms
- SSL/TLS support for secure content delivery
- Geographic load balancing with automatic failover and health-based routing
- Real-time analytics dashboard with historical data and trend analysis
- Content compression and optimization (minification, image resizing)
- Distributed cache coherence protocols for multi-region consistency

- Advanced routing algorithms (reinforcement learning, multi-armed bandit)

Project Impact:

LiteCDN demonstrates that effective CDN functionality with sophisticated routing and caching can be implemented with limited resources, making it suitable for educational purposes, small-scale deployments, and as a foundation for more advanced CDN systems. The project successfully bridges the gap between theoretical concepts and practical implementation, providing a valuable reference for understanding CDN architecture, adaptive routing strategies, and multi-tier caching mechanisms.

The modular architecture and explicit design rationale enable the system to serve as an experimental platform for evaluating alternative routing and caching algorithms under controlled conditions, fulfilling both educational and research objectives.

REFERENCES

[1] A. Vakali and G. Pallis, "Content delivery networks: Status and trends," *IEEE Internet Computing*, vol. 7, no. 6, pp. 68–74, Nov.–Dec. 2003.

[2] M. Pathan, R. Buyya, and A. Vakali, *Content Delivery Networks*. Springer, 2008.

[3] E. Nygren, R. K. Sitaraman, and J. Sun, "The Akamai Network: A platform for high-performance internet applications," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 3, pp. 2–19, 2010.

[4] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: Evidence and implications," in *Proc. IEEE INFOCOM*, 1999, pp. 126–134.

[5] S. Podlipnig and L. Böszörmenyi, "A survey of web cache replacement strategies," *ACM Computing Surveys*, vol. 35, no. 4, pp. 374–398, 2003.

[6] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, Oct. 2001.

[7] M. Calder et al., "Mapping the Expansion of Google's Serving Infrastructure," *Proc. ACM Internet Measurement Conf. (IMC)*, 2013, pp. 313–326.

[8] L. Qiu, V. N. Padmanabhan, and G. M. Voelker, "On the placement of web server replicas," in *Proc. IEEE INFOCOM*, 2001.

[9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

ACKNOWLEDGMENTS

Portions of this manuscript were refined using large language models (ChatGPT and Claude) for language editing, structural suggestions, and formatting assistance. All architectural decisions, algorithmic designs, simulations, and technical content were developed and validated by the authors.

APPENDIX

A. Use Case Diagram

Figure 5 presents the use case diagram for the LiteCDN system.

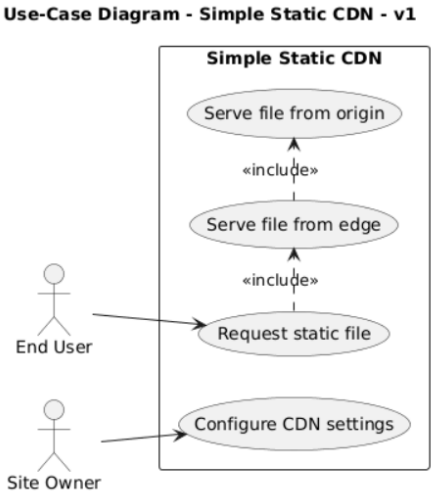

Fig. 5. Use Case Diagram — Simple Static CDN (v1)

B. Class Diagram

Figure 6 depicts the complete object-oriented structure with CDNSystem, RoutingService, EdgeServers, and OriginServer.

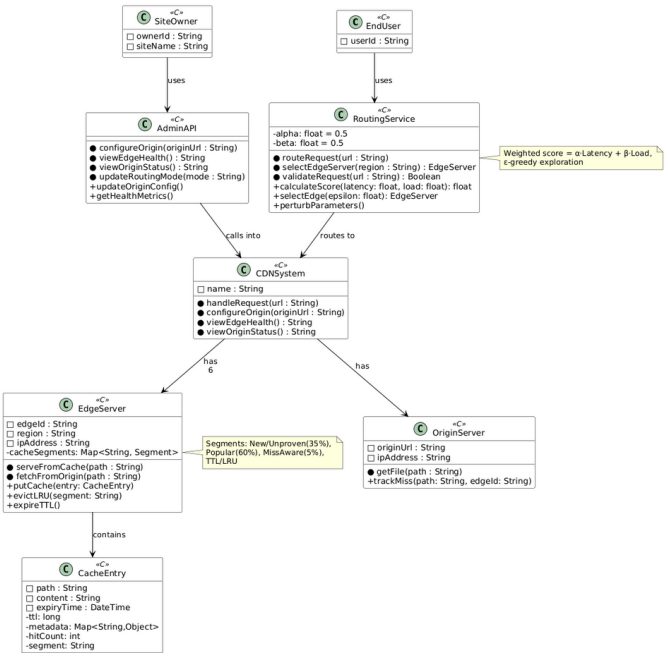

Fig. 6. Class Diagram — LiteCDN v2 (Weighted Routing + Segmented Cache)

C. Sequence Diagram

Figure 7 illustrates the request-handling sequence covering cache hit and miss scenarios.

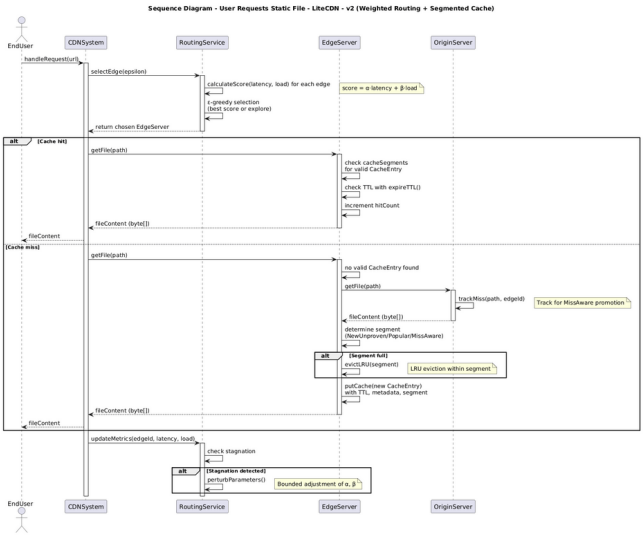

Fig. 7. Sequence Diagram — User Requests Static File, LiteCDN v2

D. Activity Diagram

Figure 8 presents the activity flow partitioned across swimlanes.

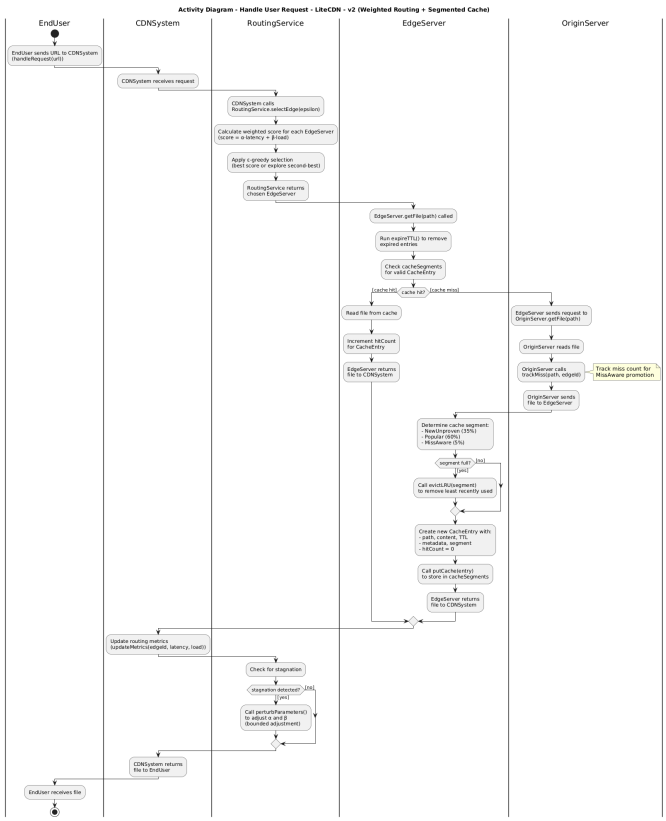

Fig. 8. Activity Diagram — Handle User Request, LiteCDN v2

E. Component Diagram

Figure 9 shows the component-level architecture.

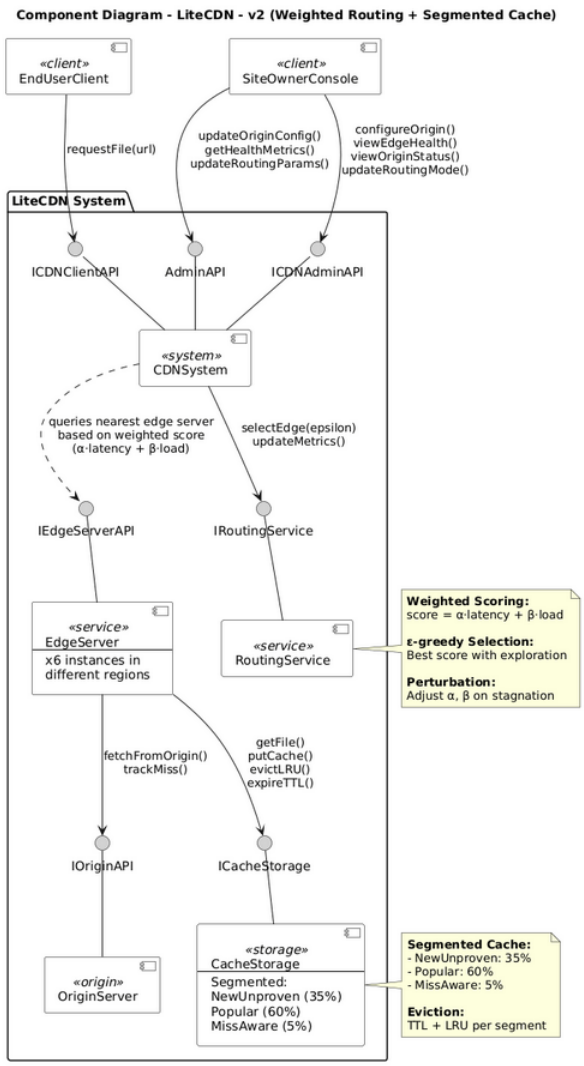

Fig. 9. Component Diagram — LiteCDN v2 (Weighted Routing + Segmented Cache)

F. Deployment Diagram

Figure 10 presents the physical deployment topology.

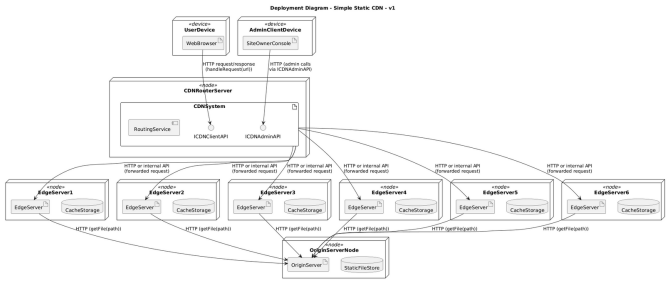

Fig. 10. Deployment Diagram — Simple Static CDN (v1)

G. Requirements Summary

Table 11 lists all tracked requirements with their types and change status.

Req ID	Requirement Name	Requirement Type	Change Type
FR-UC-01	Content Request Handling	Functional	Refined
FR-CD-01	Request Routing and Origin Fetch	Functional	Refined
FR-COMP-01	CDN Component Decomposition	Functional	Refined
FR-SEQ-01	Cache-Aware Request Flow	Functional	Refined
FR-ACT-01	Content Delivery Activity Flow	Functional	Refined
FR-DEP-01	CDN Deployment Architecture	Functional / Architectural	Refined
PR-01	Project Roles and Responsibilities	Process / Governance	New
NFR-DS-01	Design and Coding Standards	Non-Functional	New

Fig. 11. Requirements Traceability — Requirement Summary with Change Type

H. Requirements-to-Artifact Mapping

Table 12 maps each requirement to corresponding UML diagrams and verification methods.

Req ID	Requirement Description	UML / Reference Artifact	Verification Method
FR-UC-01	System shall allow an end user to request content which may be served from edge or fetched from origin	Use Case Diagram (Lab 2, Lab 3)	Use-case scenario walkthrough
FR-CD-01	System shall route client content requests to appropriate edge servers and fetch from origin if required	Class Diagram (Lab 2, Lab 3)	Class responsibility consistency review
FR-COMP-01	System shall decompose CDN functionality into routing, edge delivery, caching, and origin access components	Component Diagram (Lab 2, Lab 3)	Component interaction review
FR-SEQ-01	System shall process file requests by routing to an edge server and serving content from cache or origin	Sequence Diagram (Lab 2, Lab 3)	Sequence flow walkthrough
FR-ACT-01	System shall handle user requests by checking cache,fetching from origin on a miss, caching results, and serving content	Activity Diagram (Lab 2, Lab 3)	Activity flow consistency review
FR-DEP-01	System shall be deployed using a CDN architecture with multiple edge servers retrieving content from an origin server	Deployment Diagram (Lab 2, Lab 3)	Deployment topology review
PR-01	Project shall define roles and responsibilities to ensure architectural consistency, requirement ownership, and validation	SRS Section 2.8	Documentation review
NFR-DS-01	System implementation shall follow defined naming, commenting, and structural coding standards to ensure maintainability	SRS Section 2.9	Standards compliance review

Fig. 12. Requirements Traceability — Requirement-to-Artifact Mapping and Verification